RCAAS platform | VPS readiness audit

Report date: 29 October 2026 (Asia/Katmandu). Source revision: 239a3f1. Local repository review; no deployment or application fixes were performed.

Recommendation: address the high-priority findings before public launch. The application builds and its existing tests pass, but that does not establish production readiness.

30 findings: 13 high, 16 medium, 1 low. Fifteen defects were reproduced in isolated local checks. Remaining items are clearly labeled source-confirmed risks, gaps, or a dependency advisory. No critical public remote takeover was demonstrated.

This is a broad, risk-based code and configuration audit, not a guarantee that every bug has been found. It does not certify security, uptime or legal compliance.

Application files and real customer data were left unchanged. New files are audit artifacts only; the build regenerated ignored build output. No credentials, account records or customer attachments are included.

Scope, evidence and limitations

Indexed 286 source/test/script files (source-inventory.txt). Focused manual review: all API route families; authentication/access control; stores and persistence; local/S3 assets; booking rules and status flows; attachment handling; backup/deployment; frontend API and tour loading; booking form dates; metadata and CI. This is not a claim of line-by-line review of every component.

Existing checks: 27 client tests passed; 149 of 150 server tests passed, with one skipped; total 176 passing and one skipped. TypeScript noEmit completed successfully. ESLint: zero errors, six warnings (five img optimization warnings and one effect dependency warning). The warnings are not counted as confirmed bugs.

Production build: npm run build with NEXT_DIST_DIR=.next-new passed. The first sandboxed attempt failed fetching Google Fonts; the network-enabled retry passed. Build-time internet access to font providers is therefore a deployment dependency. Build used local default URL settings, not a validated VPS environment.

Tool versions: Node 24.13.0, npm 11.21.1, built Next 16.4.3. npm registry audit: frontend zero advisories, server one moderate advisory. Registry results are a time-specific snapshot and do not analyze custom or vendored code.

New checks: node vps-audit/reproduce.mjs, 15 confirmations; see reproductions.log. Checks use scratch data under this folder, loopback HTTP and mocks, with email/provider keys absent. They delete scratch data afterward.

Not verified: real PostgreSQL execution (the database-dependent suite is skipped), real R2 provider behavior, actual VPS Linux/nginx/TLS/firewall/systemd/PM2 configuration, production secrets, DNS, email domain verification/delivery, real hotel integrations, full rendered-page crawl, real GPU/mobile performance, accessibility screen-reader behavior, backups restored onto a separate machine, or sustained load/penetration testing. Minified vendor SDK internals were not manually audited.

Existing qa-bug-hunt report was rechecked: its Nepal date issue, outlet decline-email fragment, missing email subject space and waitlist status wording are fixed in current source; they are not repeated as open bugs. A28 describes a narrower remaining multi-timezone issue.

Priority order and reading guide

P0 before launch: A01 accountless admin, A02 upload authorization, A03 publish lifecycle, A04 restored ownership, A05/A09 fail-closed storage, A06 credential concurrency. These have local reproductions.

P1 before accepting real bookings: A15 unify or explicitly disable duplicate channels; A16 enforce one API process; A17 resolve lost writes; A18 upload quotas; A23 complete restore plan; A24 production deployment configuration. Update A26 dependency and protect attachments A27.

P2 hardening and polish: range behavior, strict calendar dates, bounded engine calls, attachment retry, async error forwarding, timezone configuration, large-project loading and content completion.

Each finding contains an exact current file/line reference, impact and prerequisites, evidence level, recommended repair and acceptance check. Source-confirmed risk means the risky implementation is visible, but the full outage/exploitation scenario was not executed.

A01 is a deliberate legacy path with an unsafe consequence for the newer permission model. A15 depends on enabling two channels for one physical resource. A16 depends on multiple API processes. A23 depends on PostgreSQL and/or an external ASSET_DIR. These conditions matter when prioritizing.

A01 | High | The shared signup code also grants unrestricted administrator access

Evidence level: Reproduced

Location: server/src/routes/auth.js:74; server/src/middleware/auth.js:27; server/src/store.js:376

Impact / trigger: Anyone given the team access code to create an editor account can instead log in without an email and obtain access across projects and admin operations. Removing their account or resetting its password does not revoke this accountless session.

Evidence: reproduce.mjs A01: POST /auth/login with the team code returned an admin JWT without sub. The normal signup path uses that same EDITOR_PASSWORD. This is a legacy compatibility path, but it defeats the current account roles.

Recommended fix: Disable accountless login in production; use separate, revocable invitations for signup. Require a real current account for every admin and project action. Rotate SESSION_SECRET after retiring legacy sessions.

Acceptance check: The signup code alone must never authenticate as admin; deleted or demoted accounts and old legacy cookies must fail authorization.

A02 | High | Staff can write into any known or invented upload staging ID

Evidence level: Reproduced

Location: server/src/routes/assets.js:90; server/src/routes/assets.js:106; server/src/access.js:45

Impact / trigger: The limited staff role can write files despite being barred from creating uploads. Any signed-in user knowing another staging ID can interfere with it. Staging IDs have no stored owner; unreferenced assets also lack ownership checks.

Evidence: A02 used a staff account to PUT a chunk to ast_other_upload without creating an upload; HTTP 200 and the bytes were written. Init and chunk routes lack assetGuard/refuseStaff. Finalize is guarded but cannot undo the staging modification.

Recommended fix: Persist upload owner and state on creation; authorize init, chunk, finalize and cleanup against that owner. Refuse staff before reading upload bodies; reject unknown IDs. Give

stored assets explicit ownership.

Acceptance check: Staff writes return 403; a second editor cannot initialize, resume, finalize or delete another editor’s unused upload.

A03 | High | Unpublish then publish cannot restore a scene to live

Evidence level: Reproduced

Location: server/src/store.js:197; server/src/store.js:212

Impact / trigger: A routinely unpublished tour cannot be published again through the normal publish flow. The request fails while the scene remains offline.

Evidence: A03: publish created @1, unpublish removed publishedVersion from the draft, and republish tried immutable @1 again and threw EEXIST. The same collision follows any published version because numbering restarts at 1.

Recommended fix: Keep a monotonic lastVersion counter independent of live status, or allocate from the highest historical version under a lock. Preserve immutable snapshots.

Acceptance check: Publish/unpublish/publish repeatedly; every attempt succeeds with a new increasing version, including after restoring older snapshots.

A04 | High | Version restore rolls back scene ownership

Evidence level: Reproduced

Location: server/src/store.js:253; server/src/store.js:259

Impact / trigger: Restoring history after a scene moves out of a shared project can transfer ownership to an earlier owner or null. The person restoring it can lose access, and the old owner may regain access.

Evidence: A04 published as owner-a, detached the scene as owner-b, then restored version 1: ownerId changed back to owner-a. Restore protects propertyId but does not preserve current ownerId.

Recommended fix: Preserve ownership and all authorization metadata from the current draft during restore; restore only editable scene content.

Acceptance check: Restore a snapshot after transfer/detach/delete-project flows and assert current ownerId and permissions remain unchanged.

A05 | High | Booking read failures become empty inventory and can destroy records

Evidence level: Reproduced

Location: server/src/reservations.js:25; server/src/reservations.js:33

Impact / trigger: A corrupt reservation file or filesystem read error makes all inventory appear available. The next mutation can replace existing booking data with a new partial list, causing data loss and double bookings.

Evidence: A05 wrote malformed JSON, readReservations returned [], then withReservations overwrote the damaged file with only a new record. The catch handles every error as an empty store.

Recommended fix: Treat only ENOENT as empty. Fail closed for invalid JSON, permission and disk failures; quarantine damaged files, alert operators, and restore from backup before accepting bookings.

Acceptance check: Malformed JSON and simulated read failures must return a controlled service error and leave the original file intact.

A06 | High | One-time reset links can be redeemed twice concurrently in file mode

Evidence level: Reproduced

Location: server/src/usersStore-file.js:164; server/src/usersStore-file.js:168

Impact / trigger: Two requests with the same reset link both succeed and race to choose the account password. File-backed role/password/reset updates can also overwrite each other because whole-file atomic writes do not serialize read-modify-write operations.

Evidence: A06 called resetPassword twice through Promise.all with one token; both returned a user. The PostgreSQL implementation uses a conditional UPDATE and is stronger for this specific operation.

Recommended fix: Serialize all mutations per account, re-read and consume the token inside the lock; preferably use transactional database updates. Never allow a stale role write to revive old credentials.

Acceptance check: Concurrent redemption has exactly one success. Concurrent password reset and role change preserve both the new password and intended role.

A07 | Medium | Concurrent first signups both become administrators

Evidence level: Reproduced

Location: server/src/usersStore-file.js:106; server/src/usersStore-pg.js:56

Impact / trigger: During first deployment, two people using the team code can both receive the initial administrator role. The first-account bootstrap is not serialized.

Evidence: A07 created two users concurrently in an empty file store; both became admin. PostgreSQL has a similar check-then-insert condition; that database path was inspected, not executed.

Recommended fix: Bootstrap the first admin explicitly through an operator command, or protect initialization with a global transaction/advisory lock and an invariant.

Acceptance check: Concurrent initial registration creates at most one admin; staff invitation must never unexpectedly become admin.

A08 | Medium | Public enquiries accept nonexistent or unpublished scene references

Evidence level: Reproduced

Location: server/src/routes/leads.js:63; server/src/routes/leads.js:71

Impact / trigger: Invalid scene requests are saved and may trigger default-team email. This creates orphan leads and lets public callers submit against draft scene IDs. Valid published scenes moved between projects are routed using their current draft association.

Evidence: A08 posted a valid guest form with not-a-real-space and received a persisted lead ID. Lookup uses getScene rather than getPublishedScene and does not reject an unresolved scene.

Recommended fix: Require a valid published scene or a valid public project entry point, derive project and scene labels from that public state, and reject unknown IDs before storage/email.

Acceptance check: Unknown/draft scenes cannot create leads; published tours route enquiries to the project shown to visitors.

A09 | High | Corrupt website state is silently treated as a new website

Evidence level: Reproduced

Location: server/src/routes/sites.js:51; server/src/routes/sites.js:355

Impact / trigger: A read failure returns null, so a subsequent draft save writes a new object and discards the prior publication, schedule and review state. The live site may disappear without a recoverable API error.

Evidence: A09 wrote invalid JSON and readSite returned null. The draft route uses (await readSite(...)) ?? {} before overwriting the file.

Recommended fix: Only ENOENT means no site. Propagate corruption and disk errors; refuse mutations until recovery and retain backups.

Acceptance check: Invalid JSON yields a controlled error and a draft save cannot replace the damaged state.

A10 | Medium | External availability response size is checked after full buffering

Evidence level: Reproduced

Location: server/src/engine.js:48

Impact / trigger: An engine endpoint can make the VPS allocate a large response despite the intended 64 KB limit. Public room-availability requests can invoke this path concurrently.

Evidence: A10 supplied a mocked oversized response and proved text() completed before rejection. Code sets a timeout but does not cap streamed response bytes.

Recommended fix: Read a bounded stream, stop and cancel once the byte limit is reached; also enforce concurrency limits and a short timeout.

Acceptance check: A response over the cap is aborted before full consumption and peak memory stays bounded under simultaneous requests.

A11 | Medium | Suffix byte ranges return incorrect file content

Evidence level: Reproduced

Location: server/src/routes/assets.js:518

Impact / trigger: Media clients asking for bytes from the end receive the beginning instead, which can break MP4 metadata probing, seeking and other streaming clients.

Evidence: A11: a 10-byte file containing 0123456789 returned 0123 for Range: bytes=-3. It should return 789 with Content-Range bytes 7-9/10.

Recommended fix: Implement suffix-length semantics and reject empty/invalid ranges; preferably use an established range parser.

Acceptance check: Verify bytes=-3, bytes=-100, bytes=3-, bytes=-0, zero-length files and large integer boundaries.

A12 | Medium | Byte ranges extending past EOF are rejected unnecessarily

Evidence level: Reproduced

Location: server/src/routes/assets.js:520

Impact / trigger: A valid request beginning inside a file but ending past EOF gets 416, creating avoidable media/download failures.

Evidence: A12: Range bytes=0-99 against a 10-byte file returned 416. A satisfiable range should clamp the end and serve bytes 0-9.

Recommended fix: Clamp the end to bytes-1 when the start is valid; return 416 when no byte is satisfiable.

Acceptance check: Check bytes=0-99 returns the complete 10-byte file as 206, while bytes=10-99 returns 416.

A13 | Medium | Booking dates are validated by shape rather than calendar validity

Evidence level: Reproduced

Location: server/src/events.js:45; server/src/stays.js:69; server/src/routes/reservations.js:406

Impact / trigger: Impossible dates can enter event bookings. Some room dates normalize inconsistently or cause addDays/toISOString failures and 500 responses.

Evidence: A13 eventProblem accepted 2026-11-31 with otherwise valid inputs. Regex YYYY-MM-DD alone cannot establish that the day exists.

Recommended fix: Use one strict calendar-date parser shared across server entry points; reject unless parsing and formatting round-trip exactly. Apply it to availability and staff marks too.

Acceptance check: Reject November 31, February 30, month 00/13 and day 00; accept valid leap days and enforce date windows.

A14 | Medium | One bad scene can break the entire studio scene list

Evidence level: Reproduced

Location: server/src/store.js:44; server/src/leadsStore.js:52; server/src/usersStore-file.js:50

Impact / trigger: Listings parse every JSON file with Promise.all. A single malformed file fails the whole list; the account scan can similarly disrupt authentication for unrelated users.

Evidence: A14 inserted one broken scene alongside a healthy scene and listScenes rejected with SyntaxError. Leads and file accounts have the same global parsing pattern, confirmed by inspection.

Recommended fix: Validate on write, isolate/quarantine individual damaged records during reads, report degraded health, and avoid exposing data outside the caller’s scope while recovering.

Acceptance check: One bad record is detected and alerted without taking unrelated accounts, projects or public listings offline.

A15 | High | Website and hotspot bookings do not share room/hall inventory

Evidence level: Reproduced

Location: server/src/events.js:22; server/src/events.js:29; server/src/stays.js:26; server/src/stays.js:38

Impact / trigger: If a venue enables both booking channels for the same physical space, both can accept overlapping confirmed bookings. This is conditional on configuring both channels; embed-first operations can avoid the trigger.

Evidence: A15 supplied a confirmed website hall booking; hallHotspotFree still returned true. Hall inventory filters different kinds. Website rooms and room hotspots likewise use separate identifiers and record kinds.

Recommended fix: Give each physical resource one inventory identifier across channels and centralize locking/availability. Until implemented, enforce one booking channel for a resource in configuration.

Acceptance check: Booking through either channel blocks overlapping requests/confirmations/marks through the other.

A16 | High | Multiple API processes bypass in-memory booking locks

Evidence level: Source-confirmed risk

Location: server/src/reservations.js:29; server/src/stats.js:52; server/src/rateLimit.js:10

Impact / trigger: PM2 cluster mode, overlapping deploys or two API instances can both read the last available room/table and overwrite reservations. Rate limits multiply with each process and reset on restart. PostgreSQL accounts do not migrate booking storage.

Evidence: Booking/stat queues and rate-limit counters are process-local Maps; all instances still write shared JSON files. No multi-process stress run was performed.

Recommended fix: Launch with exactly one API process until inventory uses database transactions/shared locks. Use shared rate limits when scaling; prevent old/new API overlap during deploy.

Acceptance check: Run a two-process last-table contention test after the storage fix; exactly one reservation succeeds and no records disappear.

A17 | High | Concurrent project/scene/site updates can lose edits and permission changes

Evidence level: Source-confirmed risk

Location: server/src/store.js:385; server/src/store.js:104; server/src/routes/sites.js:412

Impact / trigger: Atomic file replacement prevents torn JSON but not lost updates. Two editors can overwrite each other; a draft save can race a publish/scheduled publish, and two claims can both appear successful.

Evidence: These operations read current objects, change them, and write with no shared per-record lock or revision check. publishDue explicitly shares the site file with draft and review operations.

Recommended fix: Add serialization and optimistic revision checks for every mutable entity, returning 409 on stale saves; use transactions for multi-file operations and ownership claims.

Acceptance check: Race publish/save, membership changes, claims and scheduled publish; all intended updates survive or receive explicit conflicts.

A18 | High | Uploads have no total quota, expiry or ZIP expansion budget

Evidence level: Source-confirmed risk

Location: server/src/routes/assets.js:41; server/src/routes/assets.js:155

Impact / trigger: An authorized account can fill VPS disk by sending many individually permitted chunks or a highly compressed ZIP. Abandoned staging data has no automatic retention policy. Disk exhaustion affects every tenant.

Evidence: Only per-chunk limits are present. ZIP extraction writes entries without total uncompressed bytes/file-count/depth limits. No staging janitor was found. No destructive disk-fill experiment was run.

Recommended fix: Set per-upload and account quotas, extracted-byte/file/depth limits, free-space thresholds, and expired-upload cleanup. Enforce budgets while streaming, including during ZIP expansion.

Acceptance check: Small-budget tests stop oversized uploads safely, remove partial extraction, and leave capacity for normal bookings.

A19 | Medium | Concurrent chunks can overwrite each other after the offset check

Evidence level: Source-confirmed risk

Location: server/src/routes/assets.js:121; server/src/routes/assets.js:130

Impact / trigger: Duplicate/retried or overlapping requests can both see the same current offset and write to the same file. An offset-0 writer may truncate another writer’s data.

Evidence: The stat/offset check and createWriteStream are separate awaits with no per-file lock; the successful response reports the requested byte count rather than a protected committed state.

Recommended fix: Serialize chunk mutations per upload/file, atomically enforce expected offsets, and prevent finalize from running while writes are in progress.

Acceptance check: Two same-offset chunks produce one accepted write and one conflict; finalize cannot race an unfinished chunk.

A20 | Medium | Engine URL filtering does not prevent DNS-based private network access

Evidence level: Source-confirmed risk

Location: server/src/engine.js:28; server/src/engine.js:46

Impact / trigger: A studio author who configures an HTTPS hostname resolving to a private address can make the VPS contact internal services. This requires author control over availabilityUrl; it is not direct visitor URL input.

Evidence: The filter checks the hostname string rather than resolved addresses. The source itself notes this gap. Redirects are correctly refused. No internal services were contacted in this review.

Recommended fix: Allowlist vetted engine hosts, or validate and pin all resolved IPv4/IPv6 addresses and enforce egress restrictions. Apply equivalent checks to every network hop.

Acceptance check: A controlled hostname resolving to loopback/private/link-local addresses is refused without initiating the connection.

A21 | Medium | Public room availability can trigger uncapped engine requests

Evidence level: Source-confirmed risk

Location: server/src/routes/reservations.js:395; server/src/engine.js:26

Impact / trigger: Changing date/guest combinations defeats the per-URL one-minute cache. Public traffic can create many simultaneous external requests and consume VPS and provider resources.

Evidence: The room availability GET route has no rateLimiter check or shared concurrency cap; the booking submission route is separately rate limited. Bad engine responses are not cached.

Recommended fix: Rate limit availability reads, validate a bounded date/guest window, deduplicate in-flight engine calls, cap per-project concurrency and add failure caching/circuit breaking.

Acceptance check: A local fake engine sees bounded requests under a flood of varied dates; healthy callers still receive timely responses.

A22 | Medium | Attachment upload consumes its one-time key before saving the file

Evidence level: Source-confirmed risk

Location: server/src/routes/reservations.js:660; server/src/routes/reservations.js:667

Impact / trigger: If mkdir or file writing fails after reservation metadata commits, the upload key is gone and the reservation claims an attachment that cannot be downloaded. The guest cannot retry normally.

Evidence: The reservation queue saves attachment metadata and deletes the key before filesystem writes begin. This ordering is visible in the route; disk failure was not injected.

Recommended fix: Stage the attachment first and commit metadata/key consumption only after durable storage succeeds, with rollback/cleanup and retry-safe operation IDs.

Acceptance check: Inject a storage failure: the guest can retry with the original key and the inbox does not show a nonexistent attachment.

A23 | High | The backup command does not back up PostgreSQL accounts or external local assets

Evidence level: Source-confirmed gap

Location: server/scripts/backup-data.js:31; server/src/usersStore.js:7; server/src/storage.js:28

Impact / trigger: After switching accounts to PostgreSQL, restoring this archive does not restore users, password hashes or reset state. An ASSET_DIR outside DATA_DIR is also excluded in local-driver mode.

Evidence: The backup script archives only DATA_DIR and optionally uploads that archive to S3. It never dumps PostgreSQL or separately archives ASSET_DIR. This is conditional on those deployment settings.

Recommended fix: Add tested PostgreSQL backups/PITR and a separate asset backup policy. Keep off-VPS copies, encrypt/restrict them, and document a complete restore including credentials/config held in an appropriate secret store.

Acceptance check: Restore into an empty machine/database; confirm accounts, permissions, booking history, attachments, scene JSON and every scan all work.

A24 | High | VPS deployment files are absent and production defaults are permissive

Evidence level: Source-confirmed gap

Location: server/src/index.js:27; server/src/index.js:111; next.config.mts:9

Impact / trigger: There is no deploy directory despite references to it. Without explicit production environment and routing, the API enables development behavior, the browser calls localhost:4000, and the API binds all interfaces. Port conflicts silently move even in production.

Evidence: No nginx/process-manager/deployment files exist in this checkout. NODE_ENV controls production secret checks, secure cookies, demo routes and localhost CORS. API_URL defaults to localhost; listen() has no host restriction and retries another port.

Recommended fix: Create reviewed nginx/TLS and process-manager configs; bind API to loopback or firewall it, pin ports and fail fast in production. Explicitly set NODE_ENV, strong secrets, CLIENT_ORIGIN, API URL and persistent data paths. Build frontend with production URLs.

Acceptance check: Test from an external machine: only intended HTTPS ports are open, cookies are secure, API routing works, restarts retain data, and a port conflict fails visibly.

A25 | Medium | Some Express 4 async authentication handlers do not forward rejected promises

Evidence level: Source-confirmed risk

Location: server/src/middleware/auth.js:77; server/src/routes/auth.js:173

Impact / trigger: A database or account-file failure during these handlers can become an unhandled rejection instead of reaching the JSON error handler. On Node’s default handling this can terminate the API process.

Evidence: The server uses Express 4. requireAdmin and GET /session are bare async functions without try/catch or a promise wrapper, unlike most other routes. A real database-outage/process-crash test was not run.

Recommended fix: Wrap every async route/middleware in a rejection-forwarding helper or consistently catch and next(err). Test failures at all authentication lookup points.

Acceptance check: Force account lookup to reject and assert a sanitized 5xx response, no hung request, and continued API health.

A26 | Medium | Server lockfile includes a qs version flagged by npm audit

Evidence level: Registry advisory

Location: server/package-lock.json:882

Impact / trigger: The installed server dependency tree has one moderate advisory. This is not proof that the vulnerable qs.stringify comma option is reachable in this application.

Evidence: npm audit --prefix server --json reported GHSA-6pq8-jhm2-2vcw (qs.stringify comma-format TypeError DoS), affected range 6.7.0 through 6.16.0, fix available. Frontend audit reported zero known advisories at audit time.

Recommended fix: Update the affected transitive dependency through a compatible parent/lockfile update, review changes, rerun audit and application checks. Avoid blind forced major upgrades.

Acceptance check: Server audit has no remaining advisory and query parsing/API regressions still pass. Source: https://github.com/advisories/GHSA-6pq8-jhm2-2vcw

A27 | Medium | Private booking attachments are missing from git ignore rules

Evidence level: Source-confirmed gap

Location: server/src/routes/reservations.js:635; .gitignore:45

Impact / trigger: New guest PDF/Word attachments under the default data directory can be accidentally added to source control, unlike users, leads and reservations which are ignored.

Evidence: Attachments are written to DATA_DIR/attachments/<project>; .gitignore has no server/src/data/attachments/ rule. No attachment contents or credentials were copied into this report.

Recommended fix: Ignore the default attachments directory and prefer runtime DATA_DIR outside the source checkout. Inspect staged changes for private data before commits.

Acceptance check: git check-ignore confirms a sample default-path attachment is ignored; no guest files are tracked.

A28 | Medium | Tour defaults assume Nepal even when the venue uses another timezone

Evidence level: Source-confirmed gap

Location: src/features/booking/booking.ts:29; src/features/booking/HotspotBookCard.tsx:59; server/src/routes/reservations.js:427

Impact / trigger: The old visitor-local-clock issue is fixed for Nepal venues, but the hotspot form calls venueDay without the project's configured timezone. A venue elsewhere can still receive dates inconsistent with server rules.

Evidence: Source inspection: form defaults and min dates use venueDay() with the Nepal default; backend uses the published booking.timezone. Not reproduced in a browser.

Recommended fix: Expose the venue clock/timezone to the hotspot form and pass it to every date default/min helper; prefer server-returned today for public booking rules.

Acceptance check: Check venues in Kathmandu, Honolulu and Auckland around midnight with visitors in different timezones.

A29 | Medium | Tour first render waits for every published scene in the project

Evidence level: Source-confirmed risk

Location: src/app/tour/TourClient.tsx:76; src/lib/api.ts:21

Impact / trigger: A large project requires all scene documents before mounting 3D. The shared API client has no timeout, so a stalled related scene request can keep the visitor on Opening the tour indefinitely.

Evidence: TourClient fetches all matching project scenes with Promise.all before setGate ready. This is a scaling/reliability risk, not a measured first-frame regression.

Recommended fix: Load the requested starting scene first, hydrate the rest lazily, and add bounded timeouts/cancellation and useful retry states. Preserve access checks.

Acceptance check: With one sibling scene delayed or failed, the requested healthy scene still opens within the first-frame target on a real mobile device.

A30 | Low | Public marketing content still contains editorial placeholders

Evidence level: Source-confirmed gap

Location: src/features/marketing/pages/GameWorldsPage.tsx:50;
src/features/marketing/pages/PrivacyPage.tsx:48

Impact / trigger: Visitors can encounter unfinished copy and unresolved service/policy facts. This lowers trust and means the public policy text may not describe the final deployment.

Evidence: Source includes visible TBI/TBC markers for formats, licensing, address and providers. Draft policy handling is intentional, but the final content still needs owner review.

Recommended fix: Replace placeholders with verified business facts and deployment-specific policy text; hide unfinished sections until approved. Do not invent business or policy claims.

Acceptance check: Crawl rendered public pages for TBI/TBC, broken contact actions and draft content before launch.

VPS launch acceptance checklist

1. Resolve the high findings, rerun the reproduction checks as regression tests expecting the corrected behavior, then run types, lint, existing tests and a production build. The audit reproductions currently assert the presence of bugs; their current success must not be mistaken for a passing post-fix suite.

2. Pin a supported Node runtime and reproducible npm ci installs for both projects. Install frontend build dependencies before compiling. Set NODE_ENV=production explicitly for the API; generate new strong SESSION_SECRET, EMBED_TOKEN_SECRET and team/bootstrap credentials. Do not reuse local values.

3. Build with HTTPS NEXT_PUBLIC_API_URL and NEXT_PUBLIC_SITE_URL, matching CLIENT_ORIGIN. The frontend environment values are compiled into browser output: restart alone does not change them. Validate same-site cookie behavior and next/image against the production API host.

4. Put nginx/TLS in front of Next and one API process, restrict direct API exposure, set body-size/timeouts appropriate for chunk/attachment uploads, preserve correct forwarded client IP handling, and test byte ranges. Verify port conflicts cannot silently shift the service.

5. Use persistent DATA_DIR and ASSET_DIR outside release directories. Transfer ignored scans, websites and other runtime records explicitly; a git clone does not include them. Inventory every published scene variant and referenced tile after migration. Do not publish sample/draft projects accidentally.

6. Exercise owner/member/staff/anonymous flows: sign-in, reset, invite, upload/resume/finalize, edit/save/publish/unpublish/restore, website publish/schedule, enquiry, booking/conflict/confirm/cancel, guest attachment and team download. Include two simultaneous editors/guests.

7. Validate mail provider sender-domain DNS and real guest delivery, provider outages, and any SMS/WhatsApp templates. Local mock tests do not establish deliverability. Test booking links and return/analytics behavior with each real hotel engine.

8. Perform and restore off-VPS backups of database, JSON stores, attachments and scans. Monitor disk space, uptime, API errors, failed mail, booking conflicts and backup failures. Confirm log retention and private data access.

9. Check desktop and real Android/iOS devices over mobile networks, first frame and long-session memory, browser history/navigation, uploads after a disconnect, form keyboard access and reduced-motion behavior. Crawl final public URLs, canonical/sitemap links and contact actions.

10. Review the active GitHub Pages workflow and account settings if the VPS becomes the primary site: avoid an unintended second indexed copy. Resolve placeholder content and document actual

hosting/storage choices before public launch.

Evidence bundle and safe follow-up

report.pdf: this printable audit. report.md: searchable text with the same findings. findings.json: structured finding list. source-inventory.txt: indexed scope. reproduce.mjs and reproductions.log: isolated confirmations. evidence/: original build, lint, tests, types and dependency audit outputs.

Recommended follow-up is to fix findings in small reviewed batches and repeat the corresponding acceptance checks. No source changes were made as part of this reporting request.

Keep this report and reproduction script private: they describe permission weaknesses. The script creates only test accounts and test data and does not use server/.env. Do not point a future adapted test harness at production data.